SAT-Inspired Eliminations for Superposition

Petar Vukmirović[1], Jasmin Blanchette[1,2], and Marijn J.H. Heule[3]

[1] Vrije Universiteit Amsterdam, Amsterdam, the Netherlands
{p.vukmirovic,j.c.blanchette}@vu.nl
[2] Université de Lorraine, CNRS, Inria, LORIA, Nancy, France
[3] Carnegie Mellon University, Pittsburgh, USA
marijn@cmu.edu

Abstract. Optimized SAT solvers not only preprocess the clause set, they also transform it during solving as inprocessing. Some preprocessing techniques have been generalized to first-order logic with equality. In this paper, we port inprocessing techniques to work with superposition, and we strengthen preprocessing. Specifically, we look into elimination of hidden literals, variables (predicates), and blocked clauses. Our evaluation using the Zipperposition prover confirms that the new techniques usefully supplement the existing superposition machinery.

1 Introduction

The power of automated reasoning tools has greatly increased in the last few decades, thanks to procedures such as conflict-driven clause learning (CDCL) [25] for propositional logic and superposition [2] for first-order logic with equality. However, the effectiveness of these procedures crucially depends on how the input problem is represented as a clause set. For better performance, the clause set can be optimized beforehand (*preprocessing*) or during the execution of the procedure (*inprocessing*). In this paper, we lift several pre- and inprocessing techniques from propositional logic to clausal first-order logic and demonstrate their usefulness in a superposition prover.

For many years, SAT solvers have used inexpensive clause simplification techniques such as hidden literal and hidden tautology elimination [17,18] and failed literal detection [14, Sect. 1.6]. We generalize these techniques to first-order logic with equality (Sect. 3). Since generalization involves reasoning about infinite sets of literals, we propose restrictions to make them usable.

Variable elimination, based on Davis–Putnam resolution [10], has been studied in the context of both propositional logic [8,31] and quantified Boolean formulas (QBFs) [6]. The basic idea is to resolve all clauses with negative occurrences of a propositional variable (i.e., a nullary predicate symbol) against clauses with positive occurrences and delete the parent clauses. Eén and Biere [11] refined the technique to identify a subset of clauses that effectively define a variable and use it to further optimize the clause set. This latter technique, *variable elimination by substitution*, has been an important preprocessor component in many SAT solvers since its introduction in 2004.

Khasidashvili and Korovin [20] generalized variable elimination to preprocess first-order problems, yielding a technique we call *singular predicate elimination*. We extend their work along two axes (Sect. 4): We generalize Eén and Biere's refinement to first-order logic, resulting in *defined predicate elimination*, and explain how both types of predicate elimination can be used during saturation.

The last technique we study is *blocked clause elimination* (Sect. 5). It is used in both SAT [19] and QBF solvers [7]. Its generalization to first-order logic has produced good results when used as a preprocessor, especially on satisfiable problems [22]. We explore more ways to use blocked clause elimination on satisfiable problems, including using it to establish equisatisfiability with an empty clause set or as an inprocessing rule. Unfortunately, we find that its use as inprocessing can compromise the refutational completeness of superposition.

All techniques are implemented in the Zipperposition prover (Sect. 6), allowing us to ascertain their usefulness (Sect. 7). The best configuration now solves dozens of additional problems out of a randomly selected set of 4000 problems from the TPTP library [32]. The raw experimental data are publicly available.[1] More details, including detailed proofs, can be found in a technical report [34].

2 Preliminaries

Clausal First-Order Logic. Our setting is many-sorted first-order logic [15] with interpreted equality and a distinguished type o. Each variable x is assigned a non-Boolean type, and each symbol f is assigned a tuple $(\tau_1, \ldots, \tau_n, \tau)$ where $n \geq 0$, τ_i are non-Boolean types, and τ is the *result type*. We distinguish between *predicate symbols*, with o as the result type, and *function symbols*. Nullary function symbols are called *constants*. Terms are either variables x or well-typed applications $\mathsf{f}(t_1, \ldots, t_n)$, or f if $n = 0$. A term is *ground* if it contains no variables. We write $\vec{a}_n$ or $\vec{a}$ to abbreviate $(a_1, \ldots, a_n)$.

An atom is an unoriented equation. A literal is an equation $s \approx t$ or a disequation $s \not\approx t$. For every predicate symbol p, the notation $\mathsf{p}(\vec{s})$ abbreviates $\mathsf{p}(\vec{s}) \approx \top$, and $\neg\mathsf{p}(\vec{s})$ abbreviates $\mathsf{p}(\vec{s}) \not\approx \top$, where $\top$ is a distinguished constant of type o. We distinguish between *predicate literals* $(\neg)\mathsf{p}(\vec{s})$ and *functional literals*. Given a literal L, we write $\neg L$ to denote its complement. A clause C is a multiset of literals, written as $L_1 \vee \cdots \vee L_n$ and interpreted disjunctively. Clauses are often defined as sets of literals, but superposition needs multisets. Given a clause set N, $N{\downarrow_2}$ denotes the subset of its binary clauses: $N{\downarrow_2} = \{L_1 \vee L_2 \mid L_1 \vee L_2 \in N\}$.

Superposition Provers. Superposition [2] is a calculus for clausal first-order logic that extends ordered resolution [3] with equality reasoning. It is refutationally complete: Given a finite, unsatisfiable clause set, it will eventually derive the empty clause. It is parameterized by a *selection function* that influences which of a clause's literals are eligible as the target of inferences. Moreover, it is compatible with the *standard redundancy criterion*, which can be used to remove a clause C from the current clause set while preserving completeness.

[1] https://doi.org/10.5281/zenodo.4552499

The redundancy criterion relies on a well-founded order $\succ$ that compares terms, literals, or clauses. If N is ground, we can delete C if it is entailed by $\prec$-smaller clauses in N. In general, we can delete C if all of its ground instances are entailed by $\prec$-smaller ground instances of other clauses in N. The criterion can be used to delete a clause that is *subsumed* by another clause (e.g., $\mathsf{p}(\mathsf{a}) \lor \mathsf{q}$ by $\mathsf{p}(x)$) or to *simplify* a clause C into C', which amounts to adding C' and then deleting C by appealing to redundancy with respect to $N \cup \{C'\}$. Subsumption and simplification are the main inprocessing mechanisms available to superposition provers. Some systems also implement clause splitting [12, 29, 33].

Superposition provers saturate the input problem with respect to the calculus's inference rules using the *given clause procedure* [1, 26]. It partitions the proof state into a passive set $\mathcal{P}$ and an active set $\mathcal{A}$. All clauses start in $\mathcal{P}$. At each iteration of the procedure's main loop, the prover chooses a clause C from $\mathcal{P}$, simplifies it and moves it to $\mathcal{A}$. Then all inferences between C and active clauses are performed. The resulting clauses are again simplified and put in $\mathcal{P}$.

3 Hidden-Literal-Based Elimination

In propositional logic, binary clauses from a clause set N can be used to efficiently discover literals L, L' for which the implication $L' \longrightarrow L$ is entailed by N's binary clauses—i.e., $N{\downarrow_2} \models L' \longrightarrow L$. Heule et al. [18] introduced the concept of *hidden literals* to capture such implications.

Definition 1. Given a propositional literal L and a propositional clause set N, the set of *propositional hidden literals* for L and N is $\mathrm{HL_a}(L, N) = \{L' \mid L' \hookrightarrow_{\mathrm{p}}^{*} L\} \setminus \{L\}$, where $\hookrightarrow_{\mathrm{p}}$ is defined such that $\neg L' \hookrightarrow_{\mathrm{p}} L$ whenever $L' \lor L \in N$. Moreover, $\mathrm{HL_a}(L_1 \lor \cdots \lor L_n, N) = \bigcup_{i=1}^{n} \mathrm{HL_a}(L_i, N)$.

Heule et al. used a fixpoint computation, but our definition based on the reflexive transitive closure is equivalent. Intuitively, a hidden literal can be added to or removed from a clause without affecting its semantics in models of N. By removing hidden literals from C, we simplify it; by adding hidden literals to C, we may detect that $N{\downarrow_2} \models C$ and delete C. With both techniques, the resulting clause set is equivalent to N. Note that $\mathrm{HL_a}(L, N)$ is finite for a finite N.

Definition 2. Given $L' \lor L \lor C \in N$, if $L' \in \mathrm{HL_a}(L, N)$, *hidden literal elimination* (HLE) replaces N by $(N \setminus \{L' \lor L \lor C\}) \cup \{L \lor C\}$. Given $C \in N$, $\{L_1, \ldots, L_n\} = \mathrm{HL_a}(C, N)$, and $C' = C \lor L_1 \lor \cdots \lor L_n$, if C' is a tautology (i.e., $\models C'$), *hidden tautology elimination* (HTE) replaces N by $N \setminus \{C\}$.

We generalize hidden literals to first-order logic with equality by considering substitutivity of variables as well as congruence of equality.

Definition 3. Given a literal L and a clause set N, the set of *hidden literals* for L and N is $\mathrm{HL}(L, N) = \{L' \mid L' \hookrightarrow^{*} L\} \setminus \{L\}$, where $\hookrightarrow$ is defined so that (1) $\neg L'\sigma \hookrightarrow L\sigma$ if $L' \lor L \in N$ and σ is a substitution; (2) $s \approx t \hookrightarrow u[s] \approx u[t]$ for all terms s, t and contexts $u[\,]$; and (3) $u[s] \not\approx u[t] \hookrightarrow s \not\approx t$ for all terms s, t and contexts $u[\,]$. Moreover, $\mathrm{HL}(L_1 \lor \cdots \lor L_n, N) = \bigcup_{i=1}^{n} \mathrm{HL}(L_i, N)$.

The generalized definition also enjoys the key property that $L' \in \mathrm{HL}(L, N)$ implies $N{\downarrow_2} \models L' \longrightarrow L$. However, $\mathrm{HL}(L, N)$ may be infinite even for predicate literals; for example, $\mathsf{p}(\mathsf{f}^i(x)) \in \mathrm{HL}(\mathsf{p}(x), \{\mathsf{p}(x) \lor \neg\mathsf{p}(\mathsf{f}(x))\})$ for every i.

Based on Definition 3, we can generalize hidden literal elimination:

$$\frac{L' \lor L \lor C}{L \lor C}\,\mathrm{HLE} \quad \text{if } L' \in \mathrm{HL}(L, N) \qquad \frac{L \lor C}{C}\,\mathrm{FLE} \quad \text{if } L', \neg L' \in \mathrm{HL}(\neg L, N)$$

Double lines denote *simplification rules*: When the premises appear in the clause set, the prover can appeal to the redundancy criterion to replace them by the conclusions. The second rule is called *failed literal elimination*, inspired by the SAT technique of asserting $\neg L$ if L is a *failed literal* [14]. Both rules are sound.

Example 4. Consider the clause set $N = \{\mathsf{p}(x) \lor \neg\mathsf{p}(\mathsf{f}(x)),\ \mathsf{p}(\mathsf{f}(\mathsf{f}(x))) \lor \mathsf{a} \approx \mathsf{b}\}$ and the clause $C = \mathsf{f}(\mathsf{a}) \not\approx \mathsf{f}(\mathsf{b}) \lor \mathsf{p}(x)$. The first clause in N induces $\mathsf{p}(\mathsf{f}(x)) \hookrightarrow \mathsf{p}(x)$, $\mathsf{p}(\mathsf{f}(\mathsf{f}(x))) \hookrightarrow \mathsf{p}(\mathsf{f}(x))$, and hence $\mathsf{p}(\mathsf{f}(\mathsf{f}(x))) \hookrightarrow^* \mathsf{p}(x)$. Together with the second clause in N, it can be used to derive $\mathsf{a} \not\approx \mathsf{b} \hookrightarrow^* \mathsf{p}(x)$. Finally, using rule (3) of Definition 3, we derive $\mathsf{f}(\mathsf{a}) \not\approx \mathsf{f}(\mathsf{b}) \hookrightarrow^* \mathsf{p}(x)$—that is, $\mathsf{f}(\mathsf{a}) \not\approx \mathsf{f}(\mathsf{b}) \in \mathrm{HL}(\mathsf{p}(x), N)$. This allows us to remove C's first literal using HLE.

Hidden literals can be combined with unit clauses L' to remove more literals:

$$\frac{L' \quad L \lor C}{L' \quad C}\,\textsc{Unit}\mathrm{HLE} \quad \text{if } L'\sigma \in \mathrm{HL}(\neg L, N)$$

Given a unit clause $L' \in N$, the rule uses it to discharge $L'\sigma$ in $N \models L'\sigma \longrightarrow \neg L$. As a result, we have $N \models \neg L$, making it possible to remove L from $L \lor C$.

Example 5. Consider the clause set $N = \{\mathsf{p}(x) \lor \mathsf{q}(\mathsf{f}(x)),\ \neg\mathsf{q}(\mathsf{f}(\mathsf{a})) \lor \mathsf{f}(\mathsf{b}) \approx \mathsf{g}(\mathsf{c}),\ \mathsf{f}(x) \not\approx \mathsf{g}(y)\}$ and the clause $C = \neg\mathsf{p}(\mathsf{a}) \lor \neg\mathsf{q}(\mathsf{b})$. The first clause in N induces $\neg\mathsf{q}(\mathsf{f}(\mathsf{a})) \hookrightarrow \mathsf{p}(\mathsf{a})$, whereas the second one induces $\mathsf{f}(\mathsf{b}) \not\approx \mathsf{g}(\mathsf{c}) \hookrightarrow \neg\mathsf{q}(\mathsf{f}(\mathsf{a}))$. Thus, we have $\mathsf{f}(\mathsf{b}) \not\approx \mathsf{g}(\mathsf{c}) \hookrightarrow^* \mathsf{p}(\mathsf{a})$—that is, $\mathsf{f}(\mathsf{b}) \not\approx \mathsf{f}(\mathsf{c}) \in \mathrm{HL}(\mathsf{p}(\mathsf{a}), N)$. By applying the substitution $\{x \mapsto \mathsf{b},\ y \mapsto \mathsf{c}\}$ to the third clause in N, we can fulfill the conditions of $\textsc{Unit}$HLE and remove C's first literal.

Next, we generalize hidden tautologies to first-order logic.

Definition 6. A clause C is a *hidden tautology* for a clause set N if there exists a finite set $\{L_1, \ldots, L_n\} \subseteq \mathrm{HL}(C, N)$ such that $C \lor L_1 \lor \cdots \lor L_n$ is a tautology.

Example 7. In general, hidden tautologies are not redundant and cannot be deleted during saturation. Consider the unsatisfiable set $N = \{\neg\mathsf{a},\ \neg\mathsf{b},\ \mathsf{a} \lor \mathsf{c},\ \mathsf{b} \lor \neg\mathsf{c}\}$, the order $\mathsf{a} \prec \mathsf{b} \prec \mathsf{c}$, and the empty selection function. The only possible superposition inference from N is between the last two clauses, yielding the hidden tautology $\mathsf{a} \lor \mathsf{b}$ (after simplifying away $\top \not\approx \top$), which is entailed by the larger clauses $\mathsf{a} \lor \mathsf{c}$ and $\mathsf{b} \lor \neg\mathsf{c}$. If this clause is removed, the prover could enter an infinite loop, forever generating and deleting the hidden tautology.

To delete hidden tautologies during saturation, the prover could check that all the clause instances encountered along the computation of HL are $\prec$-smaller than a given hidden tautology. However, this would be expensive and seldom succeed, given that superposition creates lots of nonredundant hidden tautologies. Instead, we propose to simplify hidden tautologies using the following rules:

$$\frac{L \vee L' \vee C}{L \vee L'}\,\mathrm{HTR} \quad \text{if } \neg L' \in \mathrm{HL}(L, N) \qquad \frac{L \vee C}{L}\,\mathrm{FLR} \quad \text{if } L', \neg L' \in \mathrm{HL}(L, N)$$

We call these techniques *hidden tautology reduction* and *failed literal reduction*. Both rules are sound. As with hidden literals, unit clauses L' can be exploited:

$$\frac{L' \quad L \vee C}{L' \quad L}\,\textsc{Unit}\mathrm{HTR} \quad \text{if } L'\sigma \in \mathrm{HL}(L, N)$$

We give the simplification rules above the collective name of *hidden-literal-based elimination* (HLBE). Yet another use of hidden literals is for *equivalent literal substitution* [17]: If both $L' \in \mathrm{HL}(L, N)$ and $L \in \mathrm{HL}(L', N)$, we can often simplify $L'\sigma$ to $L\sigma$ in N if $L'\sigma \succ L\sigma$. We plan to investigate this further.

4 Predicate Elimination

For propositional logic, variable elimination [11] is one of the main pre- and in-processing techniques. It admits pure literal elimination as a special case. Khasidashvili and Korovin [20] generalized variable elimination to first-order logic with equality and demonstrated that it is effective as a preprocessor. We propose an improvement that makes this applicable in more cases and show that, with a minor restriction, it can safely be integrated in a superposition prover.

Definition 8. A predicate symbol is called *singular* (or "non-self-referential") for a clause set N if it occurs at most once in every clause contained in N.

Definition 9. Let $C = \mathsf{p}(\vec{s}_n) \vee C'$ and $D = \neg\mathsf{p}(\vec{t}_n) \vee D'$ be clauses with no variables in common. The clause $s_1 \not\approx t_1 \vee \cdots \vee s_n \not\approx t_n \vee C' \vee D'$ is a *flat resolvent* of C and D on p.

Given two clause sets M, N, predicate elimination iteratively replaces clauses from N containing the symbol p with all flat resolvents against clauses in M. Eventually, it yields a set with no occurrences of p.

Definition 10. Let M, N be clause sets and p be a singular predicate for M. Let $\rightsquigarrow$ be the following relation on clause set pairs and clause sets:

1. $(M, \{(\neg)\,\mathsf{p}(\vec{s}) \vee C'\} \uplus N) \rightsquigarrow (M, N' \cup N)$ if N' is the set that consists of all clauses (up to variable renaming) that are flat resolvents with $(\neg)\,\mathsf{p}(\vec{s}) \vee C'$ and a clause from M as premises. The premises' variables are renamed apart.
2. $(M, N) \rightsquigarrow N$ if N has no occurrences of p.

The *resolved set* $M \rtimes_{\mathsf{p}} N$ is the clause set N' such that $(M, N) \rightsquigarrow^* N'$.

The relation $\rightsquigarrow$ is confluent up to variable renaming. Thanks to the singularity constraint on M, it also terminates on finite sets because the following ordinal measure decreases: $\nu(\{D_1, \ldots, D_n\}) = \omega^{\nu(D_1)} \oplus \cdots \oplus \omega^{\nu(D_n)}$, where $\nu(D)$ counts the p's in D. For every transition $(M, \{C\} \cup N) \rightsquigarrow (M, N' \cup N)$, we have $\nu(N) = \omega^{\nu(C)} > \omega^{\nu(C)-1} \cdot |N'| = \nu(N')$.

Definition 11. Let N be a clause set and p be a singular predicate for N. Let N_{p}^{+} consist of all clauses of the form $\mathsf{p}(\vec{s}) \vee C' \in N$, let N_{p}^{-} consist of all clauses of the form $\neg\mathsf{p}(\vec{s}) \vee C' \in N$, let $N_{\mathsf{p}} = N_{\mathsf{p}}^{+} \cup N_{\mathsf{p}}^{-}$, and let $\overline{N}_{\mathsf{p}} = N \setminus N_{\mathsf{p}}$.

Definition 12. Let N be a clause set and p be a singular predicate for N. *Singular predicate elimination* (SPE) of p in N replaces N by $\overline{N}_{\mathsf{p}} \cup (N_{\mathsf{p}}^{+} \rtimes_{\mathsf{p}} N_{\mathsf{p}}^{-})$.

The result of SPE is satisfiable if and only if N is satisfiable [20, Theorem 1], justifying SPE's use in a preprocessor. However, eliminating singular predicates aggressively can dramatically increase the number of clauses. To prevent this, Khasidashvili and Korovin suggested to proceed only if $\lambda(N') \leq \lambda(N)$ and $\mu(N') \leq \mu(N)$, where $\lambda(N)$ is the number of literals in N and $\mu(N)$ is the sum for all clauses $C \in N$ of the square of the number of distinct variables in C.

Compared with the state of the art in SAT solving, this criterion is fairly restrictive. We relax it to make it possible to eliminate more predicates, within reason. Let $K_{\mathrm{tol}} \in \mathbb{N}$ be a tolerance parameter. A predicate elimination step from N to N' is allowed if $\lambda(N') < \lambda(N) + K_{\mathrm{tol}}$ or $\mu(N') < \mu(N)$ or $|N'| < |N| + K_{\mathrm{tol}}$.

SPE is effective, but an important refinement has not yet been adapted to first-order logic: variable elimination by substitution. Eén and Biere [11] discovered that a propositional variable x can be eliminated without computing all resolvents if it is expressible as an equivalence $\mathsf{x} \leftrightarrow \varphi$, where φ, the "gate," is an arbitrary formula that does not reference x. They partition a set N into a definition set G, essentially the clausification of $\mathsf{x} \leftrightarrow \varphi$, and $R = N_{\mathsf{p}} \setminus G$, the remaining clauses containing p. To eliminate x from N while preserving satisfiability, it suffices to resolve clauses from G against clauses from R, effectively substituting φ for x in R. Crucially, we do not need to resolve pairs of clauses from G or pairs of clauses from R. We generalize this idea to first-order logic.

Definition 13. Let G be a clause set, p be a predicate symbol, and $\vec{x}$ be distinct variables. The set G is a *definition set* for p if (1) p is singular for G, (2) G consists of clauses of the form $(\neg)\,\mathsf{p}(\vec{x}) \vee C'$ (up to variable renaming), (3) the variables in C' are all among $\vec{x}$, (4) all clauses in $G_{\mathsf{p}}^{+} \rtimes_{\mathsf{p}} G_{\mathsf{p}}^{-}$ are tautologies, and (5) $E(\vec{\mathsf{c}})$ is unsatisfiable, where the *environment* $E(\vec{x})$ consists of all subclauses C' of any $(\neg)\,\mathsf{p}(\vec{x}) \vee C' \in G$ and $\vec{\mathsf{c}}$ is a tuple of distinct fresh constants substituted in for $\vec{x}$.

A definition set G corresponds intuitively to a definition by cases in mathematics—e.g., $\mathsf{p}(\vec{x}) = \begin{cases} \top & \text{if } \varphi(\vec{x}) \\ \bot & \text{if } \psi(\vec{x}). \end{cases}$ Part (4) states that the case conditions are mutually exclusive (e.g., $\neg\varphi(\vec{x}) \vee \neg\psi(\vec{x})$), and part (5) states that they are exhaustive (e.g., $\nexists \vec{\mathsf{c}}.\ \neg\varphi(\vec{\mathsf{c}}) \wedge \neg\psi(\vec{\mathsf{c}})$). Given a quantifier-free formula $\mathsf{p}(\vec{x}) \leftrightarrow \varphi(\vec{x})$ with distinct variables $\vec{x}$ such that $\varphi(\vec{x})$ does not contain p, any reasonable clausification algorithm would produce a definition set for p.

Example 14. Given the formula $\mathsf{p}(x) \leftrightarrow \mathsf{q}(x) \wedge (\mathsf{r}(x) \vee \mathsf{s}(x))$, a standard clausification algorithm [27] produces $\{\neg\mathsf{p}(x) \vee \mathsf{q}(x),\ \neg\mathsf{p}(x) \vee \mathsf{r}(x) \vee \mathsf{s}(x),\ \mathsf{p}(x) \vee \neg\mathsf{q}(x) \vee \neg\mathsf{r}(x),\ \mathsf{p}(x) \vee \neg\mathsf{q}(x) \vee \neg\mathsf{s}(x)\}$, which qualifies as a definition set for p.

Definition sets gracefully generalize gates. They can be recognized syntactically for formulas such as $\mathsf{p}(\vec{x}) \leftrightarrow \bigvee_i \mathsf{q}_i(\vec{s_i})$ or $\mathsf{p}(\vec{x}) \leftrightarrow \bigwedge_i \mathsf{q}_i(\vec{s_i})$, or semantically: Condition (4) can be checked using the congruence closure algorithm, and condition (5) amounts to a propositional unsatisfiability check.

The key result about propositional gates carries over to definition sets.

Definition 15. Let N be a clause set, p be a predicate symbol, $G \subseteq N$ be a definition set for p, and $R = N_{\mathsf{p}} \setminus G$. *Defined predicate elimination* (DPE) of p in N replaces N by $\overline{N}_{\mathsf{p}} \cup (G_{\mathsf{p}} \rtimes_{\mathsf{p}} R_{\mathsf{p}})$.

Theorem 16. *The result of applying DPE to a clause set N is satisfiable if and only if N is satisfiable.*

Since there will typically be at most only a few defined predicates in the problem, it makes sense to fall back on SPE when no definition is found.

Definition 17. Let N be a clause set and p be a predicate symbol. If there exists a definition set $G \subseteq N$ for p, *portfolio predicate elimination* (PPE) on p in N replaces N with $\overline{N}_{\mathsf{p}} \cup (G_{\mathsf{p}} \rtimes_{\mathsf{p}} R_{\mathsf{p}})$, where $R = N_{\mathsf{p}} \setminus G$. Otherwise, if p is singular in N, it results in $\overline{N}_{\mathsf{p}} \cup (N_{\mathsf{p}}^{+} \rtimes_{\mathsf{p}} N_{\mathsf{p}}^{-})$. In all other cases, it is not applicable.

Hidden-literal-based techniques fit within the traditional framework of saturation, because they delete or reduce a clause based on the *presence* of other clauses. In contrast, predicate elimination relies on the *absence* of clauses from the proof state. We can still integrate it with superposition by using this strategy: At every kth iteration of the given clause procedure, perform predicate elimination on $\mathcal{A} \cup \mathcal{P}$, and add all new clauses to $\mathcal{P}$.

One may wonder whether such an approach preserves the refutational completeness of the calculus. The answer is no. To see why, consider a *binary splitting* (BS) rule [29] that replaces the premise $C \vee D$ with the conclusions $\mathsf{p} \vee C$ and $\neg\mathsf{p} \vee D$, where C and D share no free variables, p is fresh, and $\mathsf{p} \prec C, D$. This simplification rule can be repeatedly undone by predicate elimination, leading to troublesome loops: $\mathrm{BS}, \mathrm{SPE}, \mathrm{BS}, \mathrm{SPE}, \ldots$. This breaks completeness.

Our solution is to curtail the entailment relation used by the redundancy criterion to disallow splitting-like simplifications. Weak entailment $\models^\sharp$ is defined via an ad hoc nonclassical logic so that $\{\mathsf{p} \vee C,\ \neg\mathsf{p} \vee C\} \not\models^\sharp \{C\}$ and yet $\models^\sharp \{\mathsf{p} \vee \neg\mathsf{p}\}$. More precisely, this "counterintuitionistic" logic is defined via an encoding: $M \models^\sharp N$ if and only if $M^\sharp \models N^\sharp$, where $\mathsf{p}(\vec{t})^\sharp = \mathsf{p}(\vec{t}) \not\approx \bot$, $\neg\mathsf{p}(\vec{t})^\sharp = \mathsf{p}(\vec{t}) \not\approx \top$, and $L^\sharp = L$ otherwise. Moreover, the type o may be interpreted as any set of cardinality at least 2, and $\bot$ must be interpreted differently from $\top$.

The standard redundancy criterion $\mathit{Red}^\sharp$ based on $\models^\sharp$ supports all the familiar deletion and simplification techniques except splitting. Using $\mathit{Red}^\sharp$ not only prevents looping, but it also enables the use of the given clause procedure, because any redundant inference according to $\mathit{Red}^\sharp$ remains redundant after SPE or DPE. As usual, the devil is in the details, and the details are in the report [34].

5 Satisfiability by Clause Elimination

The main approaches to show satisfiability of a first-order problem are to produce either a finite Herbrand model or a saturated clause set. Saturations rarely occur except for very small problems or within decidable fragments. In this section, we explore an alternative approach that establishes satisfiability by iteratively removing clauses while preserving unsatisfiability, until the clause set has been transformed into the empty set. So far, this technique has been studied only for QBF [16]. We show that blocked clause elimination (BCE) can be used for this purpose. It can efficiently solve some problems for which the saturated set would be infinite. However, it can break the refutational completeness of a saturation prover. We conclude with a procedure that transforms a finite Herbrand model into a sequence of clause elimination steps ending in the empty clause set, thereby demonstrating the theoretical power of clause elimination.

Kiesl et al. [22] generalized *blocked clause elimination* (BCE) to first-order logic. Their generalization uses flat L-resolvents, an extension of flat resolvents.

Definition 18. Let $C = L \vee C'$ and $D = L_1 \vee \cdots \vee L_m \vee D'$, where (1) $m \geq 1$, (2) the literals L_i are of opposite polarity to L, (3) L's atom is $\mathsf{p}(\vec{s}_n)$, (4) L_i's atom is $\mathsf{p}(\vec{t_i})$ for each i, and (5) C and D have no variables in common. The clause $\big(\bigvee_{i=1}^{m} \bigvee_{j=1}^{n} s_j \not\approx t_{ij}\big) \vee C' \vee D'$ is a *flat L-resolvent* of C and D.

Definition 19. A clause $L \vee C'$ is *blocked* on L in a clause set N if all flat L-resolvents between $L \vee C'$ and clauses in N are tautologies.

Removing a blocked clause from a set preserves unsatisfiability [22]. Kiesl et al. evaluated the effect of removing all blocked clauses as a preprocessing step, and found that it increases prover's success rate.

In fact, there exist satisfiable problems that cannot be saturated in finitely many steps regardless of the calculus's parameters but that can be reduced to an empty, vacuously satisfiable problem through blocked clause elimination.

Example 20. Consider the clause set N consisting of $C = \mathsf{p}(x, x)$ and $D = \neg\mathsf{p}(y_1, y_3) \vee \mathsf{p}(y_1, y_2) \vee \mathsf{p}(y_2, y_3)$. Note that all literals are eligible for inferences regardless of the selection function. In particular, the superposition of $\mathsf{p}(x, x)$ into D's negative literal eventually needs to be performed, with the conclusion $E_1 = \mathsf{p}(z_1, z_2) \vee \mathsf{p}(z_2, z_1)$. Again, both literals are $\prec$-incomparable, and superposition of E_1 into D yields $E_2 = \mathsf{p}(z_1, z_2) \vee \mathsf{p}(z_2, z_3) \vee \mathsf{p}(z_3, z_1)$. Repeating this process yields infinitely many clauses $E_i = \mathsf{p}(z_1, z_2) \vee \cdots \vee \mathsf{p}(z_i, z_{i+1}) \vee \mathsf{p}(z_{i+1}, z_1)$ that cannot be eliminated using standard redundancy-based techniques.

In the example above, the clause D is blocked on its second and third literals. If we delete D, C becomes blocked in turn. Deleting C leaves us with the empty set, which is vacuously satisfiable. The example suggests that using BCE during saturation might help focus the proof search. Indeed, Kiesl et al. ended their investigations by asking whether BCE can be used as an inprocessing technique in a saturation prover. Unfortunately, in general the answer is no.

Example 21. Consider the unsatisfiable set $N = \{C_1, \ldots, C_6\}$ with

$$C_1 = \neg\mathsf{c} \vee \mathsf{e} \vee \boxed{\neg\mathsf{a}} \qquad C_2 = \neg\mathsf{c} \vee \boxed{\neg\mathsf{e}} \qquad C_3 = \mathsf{b} \vee \boxed{\mathsf{c}}$$
$$C_4 = \neg\mathsf{b} \vee \boxed{\neg\mathsf{c}} \qquad C_5 = \mathsf{a} \vee \boxed{\mathsf{b}} \qquad C_6 = \mathsf{c} \vee \boxed{\neg\mathsf{b}}$$

Assume the simplification ordering $\mathsf{a} \prec \mathsf{b} \prec \mathsf{c} \prec \mathsf{d} \prec \mathsf{e}$ and the selection function that chooses the last negative literal of a clause as presented. Gray boxes indicate literals that can take part in superposition inferences. Only two superposition inferences are possible: from C_3 into C_4, yielding the tautology $C_7 = \mathsf{b} \vee \boxed{\neg\mathsf{b}}$, and from C_5 into C_6, yielding $C_8 = \mathsf{a} \vee \boxed{\mathsf{c}}$. Clause C_7 is clearly redundant, whereas C_8 is blocked on its first literal. If we allow removing blocked clauses, the prover enters a loop: C_8 is generated, deleted, generated, deleted, and so on.

Although using BCE as inprocessing breaks the completeness of superposition in general, it is conceivable that a well-behaved fragment of BCE might exist. This could be investigated further.

Not only can BCE prevent infinite saturation (Example 20), but it can also be used to convert a finite Herbrand model into a certificate of clause set satisfiability. The certificate uses only blocked clause elimination and addition, in conjunction with a transformation to reduce the clause set to an empty set. This theoretical result explores the relationship between Herbrand models and satisfiability certificates based on clause elimination and addition. It is conceivable that it can form the basis of an efficient way to certify Herbrand models.

In propositional logic, *asymmetric literals* can be added to or removed from clauses, retaining the equivalence of the resulting clause set with the original one. Kiesl and Suda [21] described an extension of this technique to first-order logic. Their definition of asymmetric literals can be relaxed to allow us to add more literals, but the resulting set is then only equisatisfiable to the original one, not equivalent. This in turn allows us to show that a problem is satisfiable by reducing it to an empty problem, as is done in some SAT solvers.

For the rest of this section, we work with clausal first-order logic without equality. We use Herbrand models as canonical representatives of first-order models, recalling that every satisfiable set has a Herbrand model [13, Sect. 5.4].

Definition 22. A literal L is a *global asymmetric literal* (GAL) for a clause C and a clause set N if for every ground instance $C\sigma$ of C, there exists a ground instance $D\varrho \vee L'\varrho$ of $D \vee L' \in N \setminus \{C\}$ such that $D\varrho \subseteq C\sigma$ and $\neg L'\varrho = L\sigma$.

Theorem 23. *If L is a GAL for the clause C and the clause set N, then the set $(N \setminus \{C\}) \cup \{C \vee L\}$ is satisfiable if and only if N is satisfiable.*

For first-order logic without equality, a clause is blocked if all of its L-resolvents are tautologies [22], where the L-resolvent between $L \vee C$ and $\neg L_1 \vee \cdots \vee \neg L_n \vee D$ is $(C \vee D)\sigma$, where σ is the most general unifier [3] of the literals $L, L_1, \ldots, L_n$. Given a Herbrand model $\mathcal{J}$ of a problem, the following procedure removes all clauses while preserving satisfiability:

1. Let $\mathsf{p}(\vec{s})$ be an atom in the Herbrand universe and q be a fresh predicate symbol. If $\mathcal{J} \models \mathsf{p}(\vec{s})$, add the clause $\mathsf{q} \vee \mathsf{p}(\vec{s})$; otherwise, add $\mathsf{q} \vee \neg\mathsf{p}(\vec{s})$. Adding either clause preserves satisfiability since they are both blocked on q.

2. Since $\mathcal{J}$ is a model, for each ground instance $C\sigma$, there exists a clause $\mathsf{q} \vee L$ with $L \in C\sigma$. We can transform $C \in N$ into $C \vee \neg\mathsf{q}$, since $\neg\mathsf{q}$ is a GAL for C and N.
3. Consider the clause $\mathsf{q} \vee L$ added by step 1. Since L is ground and no clause $\mathsf{q} \vee \neg L$ was added (since $\mathcal{J}$ is a model), the only L-resolvents are against clauses added by step 2. Since all of those clauses contain $\neg\mathsf{q}$, the resolvents are tautologies. Thus, each $\mathsf{q} \vee L$ is blocked and can be removed in turn.
4. The remaining clauses all contain $\neg\mathsf{q}$. They can be removed by BCE as well.

6 Implementation

Hidden-literal-based, predicate, and blocked clause elimination can all be implemented to work efficiently in a superposition prover. In this section, we describe how to implement the first two sets of techniques. For BCE, we refer to Kiesl et al. [22]. All techniques are implemented in the Zipperposition prover [9].

Hidden-Literal-Based Elimination. For HLBE, an efficient representation of $\mathrm{HL}(L, N)$ is crucial. Because this set may be infinite, we underapproximate it by restricting the length of the transitive chains via a parameter K_{len}. Given the current clause set N, the finite map $\mathit{Imp}[L']$ associates with each literal L' a set of pairs (L, M) such that $L' \hookrightarrow^k L$, where $k \leq K_{\mathrm{len}}$ and M is the multiset of clauses used to derive $L' \hookrightarrow^k L$. Moreover, we consider only transitions of type (1) (as per Definition 3). The following algorithm maintains *Imp* dynamically, updating it as the prover derives and deletes clauses. It depends on the global variable *Imp* and the parameters K_{len} and K_{imp}.

procedure AddImplication($L_{\mathrm{a}}, L_{\mathrm{c}}, C$)
  if $\mathit{Imp}[L_{\mathrm{a}}\sigma] \neq \emptyset$ for some renaming σ **then**
    $(L_{\mathrm{a}}, L_{\mathrm{c}}) \leftarrow (L_{\mathrm{a}}\sigma, L_{\mathrm{c}}\sigma)$
  if there are no L, L', M, σ such that $(L', M) \in \mathit{Imp}[L]$, $L\sigma = L_{\mathrm{a}}$, and $L'\sigma = L_{\mathrm{c}}$ **then**
    for all $(L, M) \in \{(L_{\mathrm{a}}\sigma, M) \mid (L_{\mathrm{c}}\sigma, M) \in \mathit{Imp}[L_{\mathrm{a}}\sigma]$ for some $\sigma\}$ **do**
      remove all (L', M') such that $M \subseteq M'$ from $\mathit{Imp}[L]$
    for all L such that $(L', M) \in \mathit{Imp}[L]$ and $L_{\mathrm{a}}\sigma = L'$ for some σ **do**
      if $|M| < K_{\mathrm{len}}$ **then**
        $\mathit{Imp}[L] \leftarrow \mathit{Imp}[L] \cup (L_{\mathrm{c}}\sigma, M \uplus \{C\})$
    for all L such that $\mathit{Imp}[L] \neq \emptyset$ and $L\sigma = L_{\mathrm{c}}$ for some σ **do**
      $\mathit{Concl} \leftarrow \{(L'\sigma, M \cup \{C\}) \mid (L', M) \in \mathit{Imp}[L], |M| < K_{\mathrm{len}}\}$
      $\mathit{Imp}[L_{\mathrm{a}}] \leftarrow \mathit{Imp}[L_{\mathrm{a}}] \cup \mathit{Concl}$
    $\mathit{Imp}[L_{\mathrm{a}}] \leftarrow \mathit{Imp}[L_{\mathrm{a}}] \cup \{(L_{\mathrm{c}}, \{C\})\} \cup \{(s \not\approx t, \{C\}) \mid \exists u.\, L_{\mathrm{c}} = u[s] \not\approx u[t]\}$

procedure TrackClause(C)
  if $C = L_1 \vee L_2$ **then**
    AddImplication($\neg L_1$, L_2, C)

```
        AddImplication(¬L2, L1, C)
        if L2 = ¬L1σ for some nonidempotent σ then
20          for all i ← 1 to K_imp do
                L2 ← L2σ
                AddImplication(¬L1, L2, C)

procedure UntrackClause(C)
    for all La, Lc, M such that (Lc, M) ∈ Imp[La] and C ∈ N do
25      remove (Lc, M) from Imp[La]
```

The algorithm views a clause $L \vee L'$ as two implications $\neg L \longrightarrow L'$ and $\neg L' \longrightarrow L$. It stores only one entry for all literals equal up to variable renaming (line 2). Each implication $L_\mathrm{a} \longrightarrow L_\mathrm{c}$ represented by the clause is stored only if its generalization is not present in *Imp* (line 5). Conversely, all instances of the implication are removed (line 6).

Next, the algorithm finds each implication stored in *Imp* that can be linked to it. Either L_c becomes the new consequent (line 8) or L_a becomes the new antecedent (line 11). If L_c can be decomposed into $u[s] \not\approx u[t]$, rule (3) of Definition 3 makes it possible to store $s \not\approx t$ in $\mathit{Imp}[L_\mathrm{a}]$ (line 14). This is an exception to the idea that transitive chains should only rely on rule (1). The application of rule (3) does not count toward the bound K_len.

In first-order logic, different instances of the same clause can be used along a transitive chain. For example, the clause $C = \neg\mathsf{p}(x) \vee \mathsf{p}(\mathsf{f}(x))$ induces $\mathsf{p}(x) \hookrightarrow^i \mathsf{p}(\mathsf{f}^i(x))$ for all i. The algorithm discovers such self-implications (line 19): For each clause C of the form $\neg L \vee L\sigma$, where σ is nonidempotent, the entires $(L\sigma^2, \{C\})$, $\ldots, (L\sigma^{K_\mathrm{imp}+1}, \{C\})$ are added to $\mathit{Imp}[L]$, where K_imp is a parameter.

To track and untrack clauses efficiently, we implement the mapping *Imp* as a nonperfect discrimination tree [28]. Given a query literal L, this data structure efficiently finds all literals L' such that for some σ, $L'\sigma = L$ and $\mathit{Imp}[L'] \neq \emptyset$. We can use it to optimize all lookups except the one on line 8. For this remaining lookup, we add an index Imp^{-1} that inverts *Imp*, i.e., $\mathit{Imp}^{-1}[L] = \{L' \mid \mathit{Imp}[L'] = (L, M) \text{ for some } M\}$. To avoid sequentially going through all entries in *Imp* when the prover deletes them, for each clause C we keep track of each literal L such that C appears in $\mathit{Imp}[L]$. To implement the rules FLE and FLR, when (L', M) is added to $\mathit{Imp}[L]$, we check whether $(\neg L', M) \in \mathit{Imp}[L]$ for some M. If so, we note that there are complementary literals in $\mathrm{HL}(\neg L, M)$. The implementation of HLE, UnitHLE, HTR, and UnitHTR is straightforward.

In propositional logic, the most efficient known approach constructs the *binary implication graph* for the clause set N [18], with edges $(\neg L, L')$ and $(\neg L', L)$ whenever $L \vee L' \in N$. To avoid traversing the graph repeatedly, solvers rely on timestamps to discover connections between literals. This relies on syntactic comparisons of literals, which is very efficient in propositional logic but not in first-order logic, because of substitutions and congruence.

Predicate Elimination. To implement portfolio predicate elimination, we maintain a record for each predicate symbol p occurring in the problem with the following fields: set of definition clauses for p, set of nondefinition clauses in

which p occurs once, and set of clauses in which p occurs more than once. These records are kept in a priority queue, prioritized by properties such as presence of definition sets and number of estimated resolutions. If p is the highest-priority symbol that is eligible for SPE or DPE, we eliminate it by removing all the clauses stored in p's record from the proof state and by adding flat resolvents to the passive set. Eliminating a symbol might make another symbol eligible.

As an optimization, predicate elimination keeps track only of symbols that appear at most K_{occ} times in the clause set. For inprocessing, we use signals that Zipperposition emits whenever a clause is added to or removed from the proof state and update the records accordingly. At the beginning of the 1st, $(K_{\mathrm{iter}} + 1)$st, ... iteration of the given clause procedure's loop body, predicate elimination is systematically applied to the entire proof state. The first application of inprocessing amounts to preprocessing. By default, $K_{\mathrm{occ}} = 512$ and $K_{\mathrm{iter}} = 10$. The same ideas and limits also apply for blocked clause elimination.

Zipperposition uses its integrated SAT solver to check the condition (5) of Definition 13. During experimentation, we noticed that recognizing definitions of symbols that occur in the conjecture often harms performance. Thus, Zipperposition recognizes definitions only for nonconjecture symbols.

7 Evaluation

We measure the impact of our elimination techniques for various values of their parameters. As a baseline, we use Zipperposition's first-order portfolio mode, which runs the prover in 13 configurations of heuristic parameters in consecutive time slices. None of these configurations use our new techniques. To evaluate a given parameter value, we fix it across all 13 configurations and compare the results with the baseline.

The benchmark set consists of 4000 randomly chosen CNF and FOF TPTP 7.3.0 theorems [32] with difficulty rating (the ratio of TPTP provers that cannot solve a problem) of at least 0.2 to avoid very easy problems. The experiments were carried out on StarExec servers [30] equipped with Intel Xeon E5-2609 CPUs clocked at 2.40 GHz. The portfolio mode uses a single CPU core with a CPU time limit of 300 s. The base configuration solves 1555 problems. The values in the tables indicate the number of problems solved minus 1555. Thus, positive numbers indicate gains over the baseline. The best result is shown in bold.

Hidden-Literal-Based Elimination. The first experiments use all implemented HLBE rules. To avoid overburdening Zipperposition, we can enable an option to limit the number of tracked clauses for hidden literals. Once the limit has been reached, any request for tracking a clause will be rejected until a tracked clause is deleted. We can choose which kind of clauses are tracked: only active, only passive, or both. We also vary the maximal implication chain length K_{len} and the number of computed self-implications K_{imp}.

In Zipperposition, every lookup for instances or generalizations of $s \approx t$ must be done once for each orientation of the equation. To avoid this inefficiency, and also because hidden literals do not fully exploit congruence, we can disable

	Tracked clauses			
	250	500	1000	∞
Active	+2	+3	+5	+2
Passive	−6	−8	−9	−29
Both	+5	**+8**	+7	−26

(a) Tracking only predicate literals

	Tracked clauses			
	250	500	1000	∞
Active	−7	−8	−13	−16
Passive	−10	−13	−18	−63
Both	−5	−6	−7	−65

(b) Tracking all literals

Fig. 1. Impact of the number and kinds of tracked clauses on HLBE performance

	$K_{\text{len}} = 1$	$K_{\text{len}} = 2$	$K_{\text{len}} = 4$	$K_{\text{len}} = 8$
$K_{\text{imp}} = 0$	+5	**+8**	−12	−87
$K_{\text{imp}} = 1$	+5	+7	−16	−90
$K_{\text{imp}} = 2$	+5	+6	−16	−91

Fig. 2. Impact of the parameters K_{len} and K_{imp} on HLBE performance

		Relaxed with K_{tol}				
	K&K	0	25	50	100	200
SPE preprocessing	+24	+56	+61	+60	+58	+59
PPE preprocessing	+27	+62	+66	+63	**+65**	+64

Fig. 3. Impact of the choice of criterion on performance of predicate elimination

tracking clauses with at least one functional literal. Clauses containing functional literals can then still be simplified.

Figure 1 shows the results, without and with functional literal tracking enabled, for $K_{\text{len}} = 2$ and $K_{\text{imp}} = 0$. The results suggest that tracking functional literals is not worth the effort but that tracking predicate literals is. The best improvement is observed when both active and passive clauses are tracked. Normally DISCOUNT-loop provers [1] such as Zipperposition do not simplify active clauses using passive clauses, but here we see that this can be effective. Figure 2 shows the impact of varying K_{len} and K_{imp}, when 500 clauses from the entire proof state are tracked. These results clearly indicate that computing long implication chains is counterproductive.

Predicate and Blocked Clause Elimination. For defined predicate elimination, the number of resolvents grows exponentially with the number of occurrences of p. To avoid this expensive computation, we limit the applicability of PPE to proof states for which p is singular. According to our informal experiments, full PPE, without this restriction, generally performs less well.

Predicate elimination can be done using Khasidashvili and Korovin's criterion (K&K) or using our relaxed criterion with different values of K_{tol}. Figure 3 shows the results for SPE and PPE used as preprocessors. Our numbers corroborate Khasidashvili and Korovin's findings: SPE with K&K proves 24 more problems than the base, a 1.5% increase comparable to the 1.8% they observed.

	BCE	SPE	SPE+BCE	PPE	PPE+BCE	HLBE+ PPE+BCE
Preprocessing	+12	+61	+63	+66	+64	**+72**
Inprocessing	−15	+60	+53	+64	+56	+65

Fig. 4. Performance of predicate and blocked clause elimination

	BCE	SPE	SPE+BCE	PPE	PPE+BCE
Preprocessing	+19	+19	**+24**	+19	**+24**

Fig. 5. Performance of predicate and blocked clause elimination for establishing satisfiability

Remarkably, the number of additional proved problems more than doubles when we use our criterion, for both SPE and PPE.

Although this is not evident in Figure 3, varying K_{tol} substantially changes the set of problems solved. For example, when $K_{\text{tol}} = 0$, SPE proves 38 theorems not proved using $K_{\text{tol}} = 50$. The effect weakens as K_{tol} grows. Recognizing definition sets is useful: PPE outperforms SPE regardless of the criterion.

Performing BCE and variable elimination until fixpoint increases the performance of SAT solvers [19]. We can check whether the same holds for superposition provers. In this experiment, we use the relaxed criterion with $K_{\text{tol}} = 25$ and HLBE with the best parameters: tracking up to 500 clauses from any clause set, $K_{\text{chain}} = 2$, and $K_{\text{imp}} = 0$. We consider each technique as pre- and inprocessing.

The results are summarized in Figure 4, where the + sign denotes the combination of techniques. We confirm the results obtained by Kiesl et al. about the performance of BCE as preprocessing: It helps prove 12 more problems from our benchmark set, increasing the success rate by 0.7%, compared with 0.4% for Kiesl et al. Using BCE as an inprocessing, however, hurts performance, presumably because of its incompatibility with the redundancy criterion.

For preprocessing, the combinations SPE+BCE and PPE+BCE performed roughly on a par with SPE and PPE, respectively. This stands in contrast to the situation with SAT solvers, where such a combination usually helps. It is also worth noting that the inprocessing techniques never outperform their preprocessing counterparts. The last column indicates that hidden-literal-based elimination usefully complements predicate and blocked clause elimination.

Satisfiability by Blocked Clause Elimination. Kiesl et al. found that blocked clause elimination is especially effective on satisfiable problems. To corroborate their results and ascertain whether a combination of predicate elimination and blocked clause elimination increases the success rate, we evaluate BCE on a benchmark set consisting of 1000 randomly selected satisfiable or open status TPTP FOF and CNF problems. The hardware and CPU time limits are the same as in the experiments above. Figure 5 presents the results.

The baseline establishes the satisfiability of 371 problems. We consider only preprocessing techniques, since BCE compromises refutational completeness—a "saturation" might no longer be a saturation. We note that recognizing definition sets makes no difference on satisfiable problems. In fact, SPE and PPE solve exactly the same set of problems. The sets of problems solved by BCE and PPE differ—there are 15 problems solved by BCE and not by PPE, and vice versa.

8 Conclusion

We adapted several pre- and inprocessing elimination techniques implemented in modern SAT solvers so that they work in a superposition prover. This involved lifting the techniques to first-order logic with equality but also tailoring them to work in tandem with superposition and its redundancy criterion. Although SAT solvers and superposition provers embody radically different philosophies, we found that the lifted SAT techniques provide valuable optimizations.

We see several avenues for future work. First, the implementation of hidden literals could be extended to exploit equality congruence. Second, although block clause elimination is incomplete in general, we hope to achieve refutational completeness for a substantial fragment of it. Third, predicate and blocked clause elimination, which thrives on the absence of clauses from the proof state, could be enhanced by tagging and ignoring generated clauses that have not yet been used to subsume or simplify untagged clauses. Fourth, predicate and blocked clause elimination could be extended to work with functional literals. Fifth, more SAT techniques could be adapted, including bounded variable addition [24] and blocked clause addition [23]. Sixth, the techniques we covered could be adapted to work with other first-order calculi such as SMT and tableaux, or generalized further to work with higher-order calculi such as combinatory superposition [5] and λ-superposition [4].

Acknowledgment. We are grateful to the maintainers of StarExec for letting us use their service. Uwe Waldmann participated in the search for a counterexample to completeness of BCE as inprocessing and confirmed that Example 21 is correct. He also suggested major simplifications and helped us debug the proofs of the theorems about predicate elimination. Anne Baanen helped us define the "counterintuitionistic" logic used to disallow splitting. Ahmed Bhayat, Armin Biere, and Mathias Fleury made some useful comments on our manuscript, and Mark Summerfield suggested many textual improvements. We thank them all.

Vukmirović and Blanchette's research has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation program (grant agreement No. 713999, Matryoshka). Blanchette's research has also received funding from the Netherlands Organization for Scientific Research (NWO) under the Vidi program (project No. 016.Vidi.189.037, Lean Forward).

References

1. Avenhaus, J., Denzinger, J., Fuchs, M.: DISCOUNT: A system for distributed equational deduction. In: Hsiang, J. (ed.) RTA-95. LNCS, vol. 914, pp. 397–402. Springer (1995)
2. Bachmair, L., Ganzinger, H.: Rewrite-based equational theorem proving with selection and simplification. J. Log. Comput. **4**(3), 217–247 (1994)
3. Bachmair, L., Ganzinger, H.: Resolution theorem proving. In: Robinson, J.A., Voronkov, A. (eds.) Handbook of Automated Reasoning, vol. I, pp. 19–99. Elsevier and MIT Press (2001)
4. Bentkamp, A., Blanchette, J., Tourret, S., Vukmirović, P., Waldmann, U.: Superposition with lambdas. J. Autom. Reason. To appear
5. Bhayat, A., Reger, G.: A combinator-based superposition calculus for higher-order logic. In: Peltier, N., Sofronie-Stokkermans, V. (eds.) IJCAR 2020, Part I. LNCS, vol. 12166, pp. 278–296. Springer (2020)
6. Biere, A.: Resolve and expand. In: Hoos, H.H., Mitchell, D.G. (eds.) SAT 2004. LNCS, vol. 3542, pp. 59–70. Springer (2004)
7. Biere, A., Lonsing, F., Seidl, M.: Blocked clause elimination for QBF. In: Bjørner, N., Sofronie-Stokkermans, V. (eds.) CADE-23. LNCS, vol. 6803, pp. 101–115. Springer (2011)
8. Chatalic, P., Simon, L.: ZRES: The old Davis–Putnam procedure meets ZBDD. In: McAllester, D.A. (ed.) CADE-18. LNCS, vol. 1831, pp. 449–454. Springer (2000)
9. Cruanes, S.: Superposition with structural induction. In: Dixon, C., Finger, M. (eds.) FroCoS 2017. LNCS, vol. 10483, pp. 172–188. Springer (2017)
10. Davis, M., Putnam, H.: A computing procedure for quantification theory. J. ACM **7**(3), 201–215 (1960)
11. Eén, N., Biere, A.: Effective preprocessing in SAT through variable and clause elimination. In: Bacchus, F., Walsh, T. (eds.) SAT 2005. LNCS, vol. 3569, pp. 61–75. Springer (2005)
12. Fietzke, A., Weidenbach, C.: Labelled splitting. Ann. Math. Artif. Intell. **55**(1–2), 3–34 (2009)
13. Fitting, M.: First-Order Logic and Automated Theorem Proving. Graduate Texts in Computer Science, Springer, 2nd edn. (1996)
14. Freeman, J.W.: Improvements to Propositional Satisfiability Search Algorithms. Ph.D. thesis, University of Pennsylvania (1995)
15. Gallier, J.H.: Logic for Computer Science: Foundations of Automatic Theorem Proving. Wiley (1987)
16. Heule, M., Seidl, M., Biere, A.: A unified proof system for QBF preprocessing. In: Demri, S., Kapur, D., Weidenbach, C. (eds.) IJCAR 2014. LNCS, vol. 8562, pp. 91–106. Springer (2014)
17. Heule, M.J.H., Järvisalo, M., Biere, A.: Clause elimination procedures for CNF formulas. In: Fermüller, C.G., Voronkov, A. (eds.) LPAR-17. LNCS, vol. 6397, pp. 357–371. Springer (2010)
18. Heule, M.J.H., Järvisalo, M., Biere, A.: Efficient CNF simplification based on binary implication graphs. In: Sakallah, K.A., Simon, L. (eds.) SAT 2011. LNCS, vol. 6695, pp. 201–215. Springer (2011)
19. Järvisalo, M., Biere, A., Heule, M.: Blocked clause elimination. In: Esparza, J., Majumdar, R. (eds.) TACAS 2010. LNCS, vol. 6015, pp. 129–144. Springer (2010)
20. Khasidashvili, Z., Korovin, K.: Predicate elimination for preprocessing in first-order theorem proving. In: Creignou, N., Berre, D.L. (eds.) SAT 2016. LNCS, vol. 9710, pp. 361–372. Springer (2016)

21. Kiesl, B., Suda, M.: A unifying principle for clause elimination in first-order logic. In: de Moura, L. (ed.) CADE-26. LNCS, vol. 10395, pp. 274–290. Springer (2017)
22. Kiesl, B., Suda, M., Seidl, M., Tompits, H., Biere, A.: Blocked clauses in first-order logic. In: Eiter, T., Sands, D. (eds.) LPAR-21. EPiC Series in Computing, vol. 46, pp. 31–48. EasyChair (2017)
23. Kullmann, O.: On a generalization of extended resolution. Discr. Appl. Math. **96–97**, 149–176 (1999)
24. Manthey, N., Heule, M., Biere, A.: Automated reencoding of Boolean formulas. In: Biere, A., Nahir, A., Vos, T.E.J. (eds.) HVC 2012. LNCS, vol. 7857, pp. 102–117. Springer (2012)
25. Marques-Silva, J.P., Lynce, I., Malik, S.: Conflict-driven clause learning SAT solvers. In: Biere, A., Heule, M.J.H., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability, Frontiers in Artificial Intelligence and Applications, vol. 185, pp. 131–153. IOS Press (2009)
26. McCune, W., Wos, L.: Otter—the CADE-13 competition incarnations. J. Autom. Reason. **18**(2), 211–220 (1997)
27. Nonnengart, A., Weidenbach, C.: Computing small clause normal forms. In: Robinson, J.A., Voronkov, A. (eds.) Handbook of Automated Reasoning, vol. I, pp. 335–367. Elsevier and MIT Press (2001)
28. Ramakrishnan, I.V., Sekar, R.C., Voronkov, A.: Term indexing. In: Handbook of Automated Reasoning, vol. II, pp. 1853–1964. Elsevier and MIT Press (2001)
29. Riazanov, A., Voronkov, A.: Splitting without backtracking. In: Nebel, B. (ed.) IJCAI 2001. pp. 611–617. Morgan Kaufmann (2001)
30. Stump, A., Sutcliffe, G., Tinelli, C.: StarExec: A cross-community infrastructure for logic solving. In: Demri, S., Kapur, D., Weidenbach, C. (eds.) IJCAR 2014. LNCS, vol. 8562, pp. 367–373. Springer (2014)
31. Subbarayan, S., Pradhan, D.K.: NiVER: Non-increasing variable elimination resolution for preprocessing SAT instances. In: Hoos, H.H., Mitchell, D.G. (eds.) SAT 2004. LNCS, vol. 3542, pp. 276–291. Springer (2004)
32. Sutcliffe, G.: The TPTP problem library and associated infrastructure—from CNF to TH0, TPTP v6.4.0. J. Autom. Reason. **59**(4), 483–502 (2017)
33. Voronkov, A.: AVATAR: The architecture for first-order theorem provers. In: Biere, A., Bloem, R. (eds.) CAV 2014. LNCS, vol. 8559, pp. 696–710. Springer (2014)
34. Vukmirović, P., Blanchette, J., Heule, M.J.H.: SAT-inspired eliminations for superposition (technical report). Technical report (2021), https://matryoshka-project.github.io/pubs/satelimsup_report.pdf